

Re-Perceived Effects: A Well-Formedness Contract for Accountable Actuation

Zain Dana Harper

Seattle, WA | [ORCID 0009-0001-7175-5393](https://orcid.org/0009-0001-7175-5393)

<https://github.com/HarperZ9>

July 2026

Abstract

When an agent acts on the world, it might write a file to disk or run a command. Sometimes it moves an effector such as a motor or a valve. An effector is any part that changes the physical world. The record that the action succeeded is usually the actuator's own report, and the actuator is the part that performed the action. This note describes a small connecting piece that replaces the self-report with a re-perceived effect. Re-perceived means read back again from the world. The verdict that an action matched its intent comes from reading the world back. The actuator's account of what it did is not consulted. The connecting piece has two matching halves. Each half is an independent judging rule that runs over one shared checking step. A *gate* judges a proposed action against the abilities the agent was granted, before the action runs. An *effect* rule looks at the world, read back after the action runs, and compares it to what the action said it would do. For a file write, the rule compares the hash of the bytes on disk against the intended hash. A hash is a short fingerprint computed from the bytes. A process is judged by its return code. When the channel is a read, the rule records the observed bytes as evidence. Both rules are total: a malformed input becomes UNVERIFIABLE and never raises an error. Neither rule trusts the actuator. This is a well-formedness contract for actuation. The question it answers is whether the effect the world shows matched the intent that was admitted. The contract is not a result in robotics or control theory, and it claims nothing in any physical field. Its practical value is that it turns "the action did what it claimed" into a fact anyone can re-check. The same connecting piece also fits together with a stability-claim checker that holds control-certificate claims to the same must-fail rule.

In plain terms

Imagine you ask a helper to leave a note on the fridge. You want the note to say "buy milk." The helper walks to the kitchen and comes back, then says "done."

You now have one record that the job worked, and it came from the helper. The helper may have written the note correctly, or gotten the words wrong. The helper may also have written nothing at all and still said "done," by mistake or on purpose. You cannot tell which of these happened, because the only report you have is the helper's own word.

This is the problem. Whenever the part that does a job also certifies that the job worked, the report and the work share one author. If the work failed, the same author can still report success.

There is a simple fix here, and it works because the fridge is right there to look at. You walk to the kitchen and read the note yourself. This note calls that step re-perceiving. You perceive the result again, straight from the world.

The fix has two halves. Both run the same kind of check. The first runs before the helper acts. You check that the helper is even allowed to touch the fridge. If touching the fridge was never permitted, you stop the job before it starts. This note calls that a gate.

The second half runs after the helper acts. You read the note on the fridge and compare it to what you asked for. If the words match, the result is confirmed. Different words mean the result is refused. And sometimes there is no way to read the note back at all, say the fridge is in another building. Then the honest answer is that you cannot tell, and you report exactly that. This note calls that the effect check.

The rest of the paper does exactly this for a computer agent. For a file, reading the note back means reading the file off the disk and comparing its fingerprint to the intended one. A command is checked by the return code it left behind. In every case the judge that checks the result is separate from the actor that produced it, and the answer comes from the world.

1 The self-report problem

Plainly: the part that does a job usually also certifies that the job worked, so a failed job can still report success.

An actuator reports success. A file-writer answers “ok.” A command runner returns a status it computed on its own, and an effector controller signals that it reached its target setting. In each case the acting part is vouching for its own action. Each of these reports can be wrong. Under adversarial or buggy conditions a report can even be dishonest, and nothing downstream would notice. The gap is the same one that shows up wherever a producer certifies its own output: the certificate and the thing it certifies share an author. For actuation the fix is concrete, because the world is available to read again.

2 The seam

Plainly: two matching rules, one before the action and one after, judge it against granted abilities and against the world read back, and neither rule trusts the actor.

We express actuation accountability as two judging rules over a single checking step. The step is always the same. You perceive the result, then judge it against an independent rule. The judgment becomes a certificate anyone can re-check. The two rules have the same shape.

Gate (before execution). Given the set of granted abilities, a gate rule judges a proposed action. It asks whether this channel and this target are within what was granted. The action is admitted or refused *before* it runs, and the admission is itself a certificate.

Effect (after execution). Given the executed action’s declared intent, an effect rule judges the world *read back again*:

- **File write:** the intent carries the content that was meant to land. The rule re-reads the file off disk and hashes those bytes, then compares the on-disk hash to the intended hash. It reads VERIFIED only when the two hashes match. Any difference reads REFUTED. The actuator’s “I wrote it” is never consulted.
- **Process execution:** the return code, read back, decides the verdict. A non-zero code is REFUTED, and a small head of the captured output is carried as evidence.
- **File read:** the hash of the observed bytes is recorded as evidence of what was read.
- **Unknown channel:** UNVERIFIABLE, with a stated reason. The seam refuses to pass a channel it cannot read back. It will not assume success.

Both rules are *total*. A malformed action or perceived state yields UNVERIFIABLE and never a raised error, so a caller cannot crash the check into an accidental pass.

3 Why the verdict comes from re-perception

Plainly: the verdict is computed from the world read back, so the actor has no channel through which to relabel a failed write as a success.

The design commitment is that the verdict is computed from the world read back. The actuator’s claim is not an input to it. This has the same shape as re-derivable verification found elsewhere: content addressing [2], proof-carrying results [1], and reproducible checks [3]. Here the same idea applies to the effect of an action. Earlier work applied it to a stored file. Its practical force is that the actuator has no channel through which to relabel a failed write as a success. The bytes on disk either hash to the intended value or they do not, and the rule that checks them did not author the write.

4 Composition with control-certificate claims

Plainly: the same rule extends to stability claims, where every checked claim ships with a broken twin the checker must reject, so a pass proves the check is not empty.

The same discipline extends to a stability-claim checker. This checker tests claims of the form “this trajectory satisfies this certified property.” The load-bearing rule there is a *must-fail* fixture. Every checked claim ships with a deliberately broken twin that the checker is required to reject. So a passing check demonstrates that the check is not empty on that shape. This is the actuation version of a checker that must be shown able to fail. It holds a control-certificate claim to the same re-checkable, non-empty standard as the file-write effect.

5 Honest scope

Plainly: this note gives a well-formedness contract. It does not give a physics result. The effect rule witnesses that the effect matched the declared intent, and it says nothing about whether the intent itself was correct or safe.

This is a contract about well-formedness. It does not reach into physics. The effect rule witnesses that the bytes on disk match the declared intent, or that a process returned zero. It does *not* establish that the intent was correct. It does not show that a controller is stable in any control-theoretic sense. Nothing here says a physical maneuver was safe. Those are the actor’s responsibility, and the seam makes no statement about them. The contribution is narrow. It turns “the actuation did what it claimed” from a promise the actuator issues into a fact a third party re-derives from the world. That fact is admitted ahead of time by a capability gate, and, for the control-claim case, held to a *must-fail* rule. Prior verified-control and provenance work does not, to our knowledge, ship a sealed receipt, re-verifiable offline, that binds a re-perceived effect to a declared intent at this level. The contribution is that packaging. It does not introduce a new control method.

References

- [1] Necula, G. C. *Proof-carrying code*. POPL, 1997.
- [2] Merkle, R. C. *A digital signature based on a conventional encryption function*. CRYPTO, 1987.
- [3] Reproducible Builds Project. *Reproducible Builds: a set of software development practices*. <https://reproducible-builds.org>, 2026.